



Звіт Курсова Мовчан_Антиплагиат_123КІ.docx

05/07/2026 10:29 AM

0%

Plagiarised

100%

Unique

Quality Metrics

Readability: **40% (Середня)**

Verbosity: **15% (Низька)**

Repetition: **Низька**

Informativeness: **Висока**

Clichés detected: **0**

Writing Analysis

Sentiment: **Нейтральний**

Formality: **Формальний**

Intent: **Інформувати**

Style: **Технічний звіт / Академічна робота**

Topic: **Інформаційні технології /**

Програмування

МІНІСТЕРСТВО ОСВІТИ И НАУКИ УКРАЇНИ

ОДЕСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ імені І.І. МЕЧНИКОВА

Факультет математики, фізики та інформаційних технологій

Кафедра математичного забезпечення комп'ютерних систем

КУРСОВИЙ ПРОЕКТ

з дисципліни

«Проектування інформаційних систем»

на тему:

«Приватне детективне агентство»

студента ІІІ курсу

групи 2_____

спеціальності

123 Комп'ютерна Інженерія_ *Мовчана І.О.*

Керівник: *проф. Малахов Є. В.*_____

Захищено « ____ » _____ 202 р.

з оцінкою _____

Комісія:

(ПІБ)	(Підпис)
(ПІБ)	(Підпис)
(ПІБ)	(Підпис)

Одеса – 2025

АНОТАЦІЯ

Мета даного курсового проекту – проектування і реалізація інформаційної системи для управління роботою приватного детективного агентства. Користувачами даної системи є адміністратор та детектив.

У розробленій інформаційній системі передбачений облік працівників агентства, клієнтів, детективних справ, підозрюваних, доказів, витрат і звітів. Особливістю системи є організація супроводу детективних справ: адміністратор здійснює призначення детективів, прив'язку клієнтів до справ, контролює стан справ та їх закриття, а детектив веде призначені йому справи, додає та редагує відомості про підозрюваних, докази і витрати, а також формує звітність. Це дозволяє відстежувати хід розслідування, зберігати пов'язану з ним інформацію та впорядковувати дані в межах єдиної системи.

Основною мовою програмування при розробці системи обрано C#.

Для побудови вебінтерфейсу інформаційної системи використано технологію ASP.NET Core Razor Pages.

Архітектурно система побудована відповідно до шаблону MVC.

Для взаємодії застосунку з базою даних використано дві технології доступу до даних: Entity Framework Core та Dapper. Технологія Entity Framework Core застосована для реалізації базових операцій створення, читання, оновлення та видалення записів у таблицях БД.

Для більш складних вибірок, агрегатних обчислень і спеціалізованих SQL-запитів використано Dapper.

Для реалізації окремих елементів клієнтської взаємодії використано бібліотеку jQuery.

Важливими складовими IC є механізми безпеки, реалізовані засобами платформи .NET. У системі використано авторизацію за логіном і паролем, збереження хешованих паролів у базі даних, розмежування прав доступу за ролями користувачів, а також обмеження доступу до сторінок і функцій залежно від повноважень адміністратора чи детектива.

Результатом курсового проектування є інформаційна система управління приватним детективним агентством зі зручним користувацьким інтерфейсом, яка забезпечує ефективне ведення обліку, обробку та зберігання даних, а також підтримку основних бізнес-процесів агентства. Розроблена система сприяє підвищенню швидкості роботи персоналу, зменшенню кількості помилок при обробці інформації та покращенню контролю за виконанням детективних справ. Використання сучасних технологій програмування та систем управління базами даних забезпечує надійність, масштабованість і можливість подальшого розвитку системи відповідно до потреб користувачів.

ЗМІСТ

ВСТУП 5

1 ПОСТАНОВКА ЗАДАЧІ 7

2 ПРОЕКТУВАННЯ ІНФОРМАЦІЙНОЇ СИСТЕМИ 17

3 ІНФОРМАЦІЙНЕ МОДЕЛЮВАННЯ ПРЕДМЕТНОЇ ОБЛАСТІ 19

4 ПРОГРАМНА МОДЕЛЬ ЗАСТОСУНКУ 29

5 ВИБІР ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ ДЛЯ СТВОРЕННЯ ІНФОРМАЦІЙНОЇ СИСТЕМИ 34

6 СТВОРЕННЯ БАЗИ ДАНИХ 36

7 ЗАПИТИ ДО БАЗИ ДАНИХ ДЛЯ ВИРІШЕННЯ ПОСТАВЛЕНИХ ЗАДАЧ 38

7.1 Задачі адміністраторів 38

7.2 Задачі детективів 43

7.3 Створення тригерів 46

7.4 Створення процедур 47

7.5 Створення представлень 48

8 ПРОГРАМНА РЕАЛІЗАЦІЯ ІНТЕРФЕЙСУ 49

9 БЕЗПЕКА ІНФОРМАЦІЙНОЇ СИСТЕМИ 54

10 ІНСТРУКЦІЯ КОРИСТУВАЧА 58

10.1 Інтерфейс адміністратора 58

10.2 Інтерфейс детектива 63

ВИСНОВКИ 67

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ 69

ВСТУП

Робота детективного агентства пов'язана з обробкою значного обсягу інформації, яка стосується клієнтів, працівників, детективних справ, підозрюваних, доказів, витрат і звітів. У процесі діяльності виникає потреба не лише в накопиченні цих даних, а й у їх постійному оновленні, впорядкуванні та контролі. Виконання таких дій вручну є складним, потребує значних витрат часу та створює ризик помилок, пов'язаних із людським фактором.

Специфіка роботи детективного агентства вимагає чіткого розподілу обов'язків між працівниками. Адміністратор повинен мати можливість організовувати роботу системи, реєструвати персонал, контролювати стан справ, призначати детективів і завершувати справи. Детектив, у свою чергу, повинен працювати лише з тими справами, які йому призначені, вносити інформацію про підозрюваних, докази, витрати та формувати звіти. Усе це потребує створення спеціалізованого програмного засобу, який забезпечить зручну взаємодію з даними та відповідатиме реальним потребам користувачів.

Існуючі програмні продукти здебільшого орієнтовані на загальний облік даних або на інші сфери діяльності, тому не завжди враховують особливості організації роботи детективного агентства. Часто такі системи або мають надлишкову функціональність, або, навпаки, не надають необхідних можливостей для супроводу справ, контролю доступу та ведення звітності. Саме тому розробка окремої інформаційної системи для детективного агентства є доцільною та актуальною.

Метою даного курсового проєкту є проєктування та реалізація інформаційної системи для управління роботою приватного детективного агентства. Створювана система повинна забезпечити зручний доступ до необхідної інформації, автоматизувати основні робочі процеси та підвищити ефективність роботи користувачів. Адміністратор у межах системи отримає інструмент для реєстрації персоналу, призначення детективів, прив'язки клієнтів до справ, контролю за перебігом розслідувань і закриття справ. Детектив отримає можливість вести призначені йому справи, працювати з інформацією про підозрюваних, додавати та змінювати докази, фіксувати витрати та формувати звітність.

Для досягнення поставленої мети необхідно виконати аналіз предметної області, визначити категорії користувачів і сформулювати їхні вимоги до системи, обрати архітектуру і шаблон проєктування, спроектувати базу даних, визначити технології та засоби реалізації, створити базу даних і програмний застосунок для роботи з основними сутностями предметної області, а також забезпечити цілісність, безпеку даних, захист від несанкціонованого доступу та розмежування повноважень між різними категоріями користувачів.

1 ПОСТАНОВКА ЗАДАЧІ

Завдання полягає в тому, щоб розробити інформаційну систему, яка буде використовуватися для управління приватною детективною діяльністю. Ця система повинна надавати інструменти для ефективного ведення справ і організації робочих процесів, де працівники можуть координувати розслідування, відстежувати справи, управляти інформацією про клієнтів, підозрюваних та докази. Також цю інформаційну систему можна використовувати для найбільш ефективного підбору персоналу для конкретного завдання.

Основні вимоги до ІС.

Облік клієнтів, детективів та підозрюваних: система повинна надавати необхідні дані для ведення профілів клієнтів, детективів та підозрюваних, включаючи особисту інформацію, контактні дані, а також історію справ і запитів.

Облік розслідувань: система надає всі необхідні дані для відстеження прогресу кожного розслідування, зокрема дати початку, завершення, стадії розслідування та призначених детективів.

Облік доказів: система повинна зберігати дані про докази, пов'язані з конкретними справами, їхній тип, дату отримання та місце зберігання, що забезпечить швидкий доступ та перевірку автентичності.

Моніторинг і аналіз розслідувань: система надає можливість моніторингу ходу розслідувань та аналізу зібраної інформації, що дозволяє детективам переглядати ключові події, оцінювати прогрес і формувати подальші кроки на основі отриманих даних.

Планування та управління завданнями: система забезпечує інструменти для планування та розподілу завдань між детективами, відстеження дедлайнів, що дозволяє ефективно керувати ресурсами та контролювати хід розслідувань.

Система розроблена для використання адміністраторами та детективами.

Адміністратор – має функції управління клієнтською базою, базою детективів та адміністрування справ (створення і закриття), що відповідає його ролі в системі. Крім того, адміністратору надається можливість переглядати загальний статус справ без доступу до їх деталей. Усі дії адміністратора зосереджені виключно на загальному управлінні та підтримці бази клієнтів і детективів, що забезпечує чітке розмежування його обов'язків у межах інформаційної системи.

Детектив – має повний доступ до внутрішніх деталей справ, що включає додавання, редагування та видалення звітів, доказів і підозрюваних. Ці функції є ключовими для виконання завдань у межах розслідування. Функціонал перегляду, оновлення та видалення доказів і даних підозрюваних закріплений за детективом, що відповідає його основній ролі в розслідувальній діяльності.

Таблиця 1.1 – Список задач користувачів ІС «Приватне детективне агентство»

№

Задача

Вхідні дані

Вихідні дані

1. Адміністратор

A1

Додати клієнта

ПІБ, номер телефону або email, дата народження, адреса, дата реєстрації

Новий клієнт

A2

Видалити клієнта

ID клієнта

Клієнт видалений

A3

Переглянути профіль клієнта

ID клієнта

ПІБ, номер телефону або email, адреса, описи справ, статуси

A4

Оновити інформацію про клієнта

ID клієнта, нові дані (ПІБ, номер телефону, email, дата народження, адреса)

Оновлений профіль клієнта

Продовження таблиці 1.1

№

Задача

Вхідні дані

Вихідні дані

A5

Додати детектива

ПІБ, номер телефону або email, дата народження, адреса, зарплата, нотатки, дата прийому на роботу (необов'язково)

Новий детектив

A6

Видалити детектива

ID детектива

Детектив видалений

A7

Переглянути профіль детектива

ID детектива

ПІБ, номер телефону, email, дата народження, адреса, зарплата, персональні нотатки, кількість справ, кількість завершених справ, статус

A8

Оновити інформацію детектива

ID детектива, нові дані (ПІБ, номер телефону або email, дата народження, адреса, зарплата, персональні нотатки, дата найму, статус)

Оновлений профіль детектива

A9

Створити справу

ID клієнта, тип справи, назва, опис справи, дедлайн справи, статус «відкрито»

Нова справа створена

A10

Призначити детектива до справи

ID детектива, ID справи

Детектив призначений

Продовження таблиці 1.1

№

Задача

Вхідні дані

Вихідні дані

A11

Видалити детектива зі справи

ID справи

Детектив видалений зі справи

A12

Переглянути інформацію про справу

ID справи

Статус, відповідальний детектив (ПІБ)

A13

Оновити інформацію про справу

ID справи, новий дедлайн

Дедлайн змінено

A14

Створити новий тип справи

Назва типу, ціна

Створено новий тип справи

A15

Видалити тип справи

ID типу справи

Тип справи видалено

A16

Оновити тип справи

ID типу справи, нові дані (назва чи ціна)

Тип справи оновлено

A17

Переглянути список звітів до справи

ID справи

ID справи, ID детективу, дата звіту, текст звіту, коментарі

A18

Переглянути звіт

ID звіту

ID справи, ID детективу, дата звіту, текст звіту, коментарі

A19

Переглянути перелік справ за статусом

ID статусу

ID справи, назва, ID детектива

A20

Переглянути справи, найближчі до делайну

-

ID справи, назва, ID детектива

Продовження таблиці 1.1

№

Задача

Вхідні дані

Вихідні дані

A21

Переглянути відкриті справи, найближчі до делайну

Статус справи «відкрита»

ID справи, тип справи, назва, опис справи, дедлайн справи, ПІБ детектива

A22

Переглянути

перелік справ клієнта

ID клієнта

ID справи, назва, ПІБ детектива, тип справи, дата початку справи, дата дедлайну, дата закриття справи, статус справи

A23

Переглянути відсортований список доданих клієнтів за часом додавання

Початкова дата, кінцева дата

ID клієнта, ПІБ, дата реєстрації

A24

Переглянути навантаження детектива

ID детектива

ID справи, назва, дата дедлайну

A25

Переглянути рейтинг детективів по кількості завершених справ

ID детектива

ПІБ детектива, кількість справ, відсоток успішності, середній термін виконання

A26

Переглянути список справ із простроченими дедлайнами

•

ID справи, назва, дата початку, дедлайн, статус, ПІБ детектива

Продовження таблиці 1.1

№

Задача

Вхідні дані

Вихідні дані

A27

Переглянути незвершені справи клієнта

-

ID клієнта, ПІБ, кількість справ

A28

Отримати інформацію про середню вартість справ, які вів конкретний детектив, з урахуванням основної вартості та додаткових витрат

ID детектива

ПІБ детектива, середня вартість справ

A29

Отримати зведений рейтинг детективів за поточний рік

-

ПІБ детектива, відсоток закритих справ, кількість незакритих справ, ціна усіх справ (разом), кількість доданих доказів, кількість доданих підозрюваних

A30

Отримати клієнтів, які звернулися вперше у поточному році

-

ПІБ клієнта, дата першої справи клієнта

A31

Переглянути рейтинг найпопулярніших типів справ

-

Місце у рейтингу, Назва типу справи, кількість справ

A32

Отримати рейтинг клієнтів за кількістю незавершених справ

Статус «не закрита»

Місце у рейтингу, ID клієнта, ПІБ, кількість незавершених справ

Продовження таблиці 1.1

A33

Рейтинг: які типи справ найчастіше залишаються незавершеними

Статус «не закрита»

Місце у рейтингу, назва типу справи, кількість незавершених справ

A34

Переглянути рейтинг справ за витратами

-

Місце у рейтингу, ID справи, назва справи, ПІБ клієнта, ПІБ детектива, кількість витрат

2. Детектив

Д1

Переглянути інформацію про справу

ID справи

Назва, опис, дата початку справи, статус, список підозрюваних,

Д2

Закрити справу

ID справи, статус закрито

Справа закрита

Д3

Переглянути інформацію про клієнта

ID клієнта

ПІБ, email, телефон, дата народження, адреса, дата реєстрації

Д4

Створити доказ

Опис доказу, тип, дата збору, ID справи (необов'язково)

Доказ створено

Д5

Оновити інформацію про доказ

ID доказу, новий опис

Оновлена інформація про доказ

Д6

Призначити доказ до справи

ID справи, ID доказу

Доказ додано до справи

Продовження таблиці 1.1

№

Задача

Вхідні дані

Вихідні дані

Д7

Видалити доказ зі справи

ID доказу, ID справи

Доказ видалений

Д8

Переглянути список доказів до справи

ID справи

ID доказу, тип доказу, опис, дата збору, регіон збору, статус

Д9

Створити підозрюваного

Відомі дані (ПІБ, номер телефону або email, дата народження, адреса, опис зовнішності, зріст, вага, попередні затримання)

Новий підозрюваний доданий до справи

Д10

Додати підозрюваного до справи

ID справи, ID підозрюваного

Новий підозрюваний доданий до справи

Д11

Видалити підозрюваного зі справи

ID підозрюваного

Підозрюваний видалений зі справи

Д12

Видалити підозрюваного

ID підозрюваного

Підозрюваний видалений

Д13

Переглянути профіль підозрюваного

ID підозрюваного

ПІБ, номер телефону, email, дата народження, адреса, опис зовнішності, зріст, вага, попередні затримання

Продовження таблиці 1.1

№

Задача

Вхідні дані

Вихідні дані

Д14

Оновити інформацію про підозрюваного

ID підозрюваного, нові дані (ПІБ, номер телефону або email, дата народження, адреса, опис зовнішності, зріст, вага, попередні затримання)

Оновлений профіль підозрюваного

Д15

Переглянути список підозрюваних до справи

ID справи

ID підозрюваного, ПІБ, псевдонім, телефон, місто проживання, статус

Д16

Переглянути усі справи за типом

ID типу справи

ID справи, назва справи, клієнт, детектив, дата відкриття справи, статус

Д17

Створити звіт до справи

ID справи, текст звіту

Новий звіт доданий до справи

Д18

Передивитись дедлайн справи

ID справи

Назва справи, дата початку, дата дедлайну

Д19

Список незакритих справ

Статус «незакрита»

ID справи, назва, дата початку, статус, дедлайн

Д20

Список

підозрюваних із заданими характеристиками

Зріст, вага, фізичний опис

ID підозрюваного, ПІБ, псевдонім, телефон, дата народження, місто проживання, статус

Продовження таблиці 1.1

№

Задача

Вхідні дані

Вихідні дані

Д21

Список

підозрюваних із заданими характеристиками

Зріст, вага, фізичний опис

ID підозрюваного, ПІБ, псевдонім, телефон, дата народження, місто проживання, статус

Д22

Отримати останній створений звіт у справі

ID справи

ID звіту, дата створення, короткий опис

Д23

Пошук

підозрюваних із зазначеним місцем проживання

Місто або регіон

ID підозрюваного, ПІБ, адреса

Д24

Перевірити статус усіх справ, до яких прив'язаний підозрюваний

ID підозрюваного

ID справи, назва, статус

2 ПРОЕКТУВАННЯ ІНФОРМАЦІЙНОЇ СИСТЕМИ

2.1 Архітектура ІС

Для реалізації поставлених завдань обрано трирівневу архітектуру (рис. 2.1), оскільки інформаційна система детективного агентства базується на роботі з великою кількістю взаємопов'язаних даних і потребує чіткого розподілу функціональності між окремими рівнями. У такій системі доцільно виділити три основні шари.

Рівень представлення – містить засоби користувацького інтерфейсу, за допомогою яких адміністратор і детектив взаємодіють із системою, переглядають дані, вводять нову інформацію та виконують передбачені для них операції.

Рівень бізнес-логіки – містить програмні компоненти, у яких реалізовано правила роботи інформаційної системи, обробку дій користувачів, логіку ведення справ, призначення детективів, обліку доказів, витрат і формування звітності.

Рівень даних – містить засоби зберігання та доступу до даних, збереження інформації про працівників агентства, клієнтів, справи, підозрюваних, докази, витрати та звіти.

Рисунок 2.1 – Трирівнева архітектура клієнт-сервер

2.2 Шаблон проектування

У якості шаблону проектування обрано MVC (рис. 2.2). Вибір цього шаблону зумовлений тим, що він забезпечує зручну організацію структури веб-застосунку, спрощує розробку, тестування, внесення змін і пошук помилок.

Крім того, використання шаблону MVC є доцільним у зв'язку з тим, що в межах курсового проєкту розробляється веб-застосунок, а дана модель є однією з найпоширеніших і найзручніших для веб-розробки. При використанні шаблону MVC інформаційна система поділяється на три основні складові.

1. Модель – відповідає за роботу з даними предметної області, їх збереження, зміну та передачу до інших компонентів системи. У межах даного проєкту модель забезпечує роботу з даними про працівників агентства, клієнтів, справи, підозрюваних, докази, витрати та звіти.
2. Представлення – відповідає за відображення даних користувачеві та надання інтерфейсу для взаємодії із системою. Саме через представлення адміністратор і детектив переглядають інформацію, вводять нові дані та виконують доступні їм дії.
3. Контролер – обробляє дії користувача, отримані через інтерфейс, і перетворює їх на відповідні операції над моделлю. У системі детективного агентства контролер забезпечує обробку запитів, пов'язаних із веденням справ, призначенням детективів, додаванням доказів, обліком витрат і формуванням звітності.

Рисунок 2.2 – Шаблон MVC

3 ІНФОРМАЦІЙНЕ МОДЕЛЮВАННЯ ПРЕДМЕТНОЇ ОБЛАСТІ

3.1 Опис сутностей предметної області

Опис сутностей предметної області «Приватне детективне агенство» наведений в табл. 3.1.

Таблиця 3.1 – Опис сутностей предметної області «Приватне детективне агенство»

Ім'я атрибута

Призначення атрибута

Обмеження

Case (Справа)

id

Ідентифікатор справи

Первинний ключ

case_type_id

Ідентифікатор типу справи

Зовнішній ключ для зв'язку з сутністю case_type (id), не порожнє

client_id

Ідентифікатор клієнта

Зовнішній ключ для зв'язку з сутністю client (id), не порожнє

detective_id

Ідентифікатор детектива

Зовнішній ключ для зв'язку з сутністю detective (id), може бути порожнім

title

Назва справи

Не порожнє

description

Опис справи

Не порожнє

start_date

Дата початку справи

Не порожнє, значення за замовчуванням – поточна дата

deadline_date

Кінцевий термін виконання

Не порожнє, значення $\geq$ start_date

close_date

Дата закриття справи

Може бути порожнім, якщо задано — значення $\geq$ start_date

status

Статус справи

Не порожнє, значення за замовчуванням 'Відкрито'

Продовження таблиці 3.1

Ім'я атрибута

Призначення атрибута

Обмеження

Detective (Детектив)

id

Ідентифікатор детектива

Первинний ключ

first_name

Ім'я

Не порожнє, лише українські літери

last_name

Прізвище

Не порожнє, лише українські літери

father_name

По батькові

Може бути порожнім, якщо задано — лише українські літери

phone_number

Номер телефону

Не порожнє, формат +380XXXXXXXXXX

email

Електронна пошта

Не порожнє, формат email

date_of_birth

Дата народження

Не порожнє, значення $\leq$ CURRENT_DATE

region

Область проживання

Не порожнє, лише українські літери

city

Місто проживання

Не порожнє, лише українські літери та дефіс

street

Вулиця

Не порожнє, українські літери, пробіли та дефіс

building_number

Номер будинку

Не порожнє, формат: цифри та символ /

apartment_number

Номер квартири

Може бути порожнім, якщо задано — значення > 0

hire_date

Дата прийняття на роботу

Не порожнє, значення за замовчуванням – поточна дата та час

salary

Заробітна плата

Не порожнє, значення ≥ 0

Продовження таблиці 3.1

Ім'я атрибута

Призначення атрибута

Обмеження

status

Статус детектива

Не порожнє, значення за замовчуванням 'Активний(а)'

personal_notes

Особисті примітки

Може бути порожнім

postgres_login

Акаунт детектива у БД

Не порожнє, англійські літери та цифри

Client (Клієнт)

Id

Ідентифікатор клієнта

Первинний ключ

apartment_number

Номер квартири

Може бути порожнім, якщо задано — значення > 0

building_number

Номер будинку

Не порожнє, формат: цифри та символ /

city

Місто проживання

Не порожнє, лише українські літери та дефіс

date_of_birth

Дата народження

Не порожнє, значення <= поточна дата

email

Електронна пошта

Не порожнє, формат email

father_name

По батькові

Може бути порожнім, якщо задано — лише українські літери

first_name

Ім'я

Не порожнє, лише українські літери

last_name

Прізвище

Не порожнє, лише українські літери

phone_number

Номер телефону

Не порожнє, формат +380XXXXXXXXX

region

Область

Не порожнє, лише українські літери

Продовження таблиці 3.1

Ім'я атрибута

Призначення атрибута

Обмеження

registration_date

Дата реєстрації

Не порожнє, значення за замовчуванням поточна дата та час, <= поточна дата та час

street

Вулиця

Не порожнє, українські літери, пробіли та дефіс

Evidence (Доказ)

id

Ідентифікатор доказу

Первинний ключ

description

Опис доказу

Не порожнє

type

Тип доказу

Не порожнє

collection_date

Дата і час збору доказу

Не порожнє, значення <= поточна дата та час

region

Регіон збору доказу

Не порожнє, значення за замовчуванням 'Не вказано'

annotation

Примітка до доказу

Може бути порожнім

purpose

Призначення доказу

Може бути порожнім

approval_status

Статус погодження

Не порожнє, значення за замовчуванням 'Чернетка'

created_by_detective_id

Ідентифікатор детектива, який створив доказ

Зовнішній ключ для зв'язку з сутністю detective (id), не порожнє

CaseSuspect (Підозрюваний у справі)

suspect_id

Ідентифікатор підозрюваного

Зовнішній ключ для зв'язку з сутністю suspect (id), не порожнє

case_id

Ідентифікатор справи

Зовнішній ключ для зв'язку з сутністю case (id), не порожнє

is_interrogated

Ознака проведення допиту

Не порожнє

alibi

Алібі підозрюваного

Може бути порожнім

Продовження таблиці 3.1

Ім'я атрибута

Призначення атрибута

Обмеження

CaseEvidence (Доказ у справі)

evidence_id

Ідентифікатор доказу

Зовнішній ключ для зв'язку з сутністю evidence (id), не порожнє

case_id

Ідентифікатор справи

Зовнішній ключ для зв'язку з сутністю case (id), не порожнє

Первинний ключ: (evidence_id, case_id)

CaseType (Тип справи)

id

Ідентифікатор типу справи

Первинний ключ

name

Назва типу справи

Не порожнє, українські літери, цифри та розділові знаки

price

Вартість типу справи

Не порожнє, вартість справи > 0

Expense (Витрата)

id

Ідентифікатор витрати

Первинний ключ

case_id

Ідентифікатор справи

Зовнішній ключ для зв'язку з сутністю case (id), не порожнє

date_time

Дата і час витрати

Не порожнє, значення <= поточна дата та час

purpose

Призначення витрати

Не порожнє, формат: українські та латинські літери, цифри, пробіли й символи - . , ; '

amount

Сума витрати

Не порожнє, значення > 0

annotation

Примітка до витрати

Може бути порожнім

approval_status

Статус погодження витрати

Не порожнє, значення за замовчуванням 'Чернетка'

Продовження таблиці 3.1

Ім'я атрибута

Призначення атрибута

Обмеження

created_by_detective_id

Ідентифікатор детектива, який створив витрату

Зовнішній ключ для зв'язку з сутністю detective (id), не порожнє

Report (Звіт)

id

Ідентифікатор звіту

Первинний ключ

case_id

Ідентифікатор справи

Зовнішній ключ для зв'язку з сутністю case (id), не порожнє

report_date

Дата створення звіту

Не порожнє, значення за замовчуванням поточна дата та час <= поточна дата та час

summary

Короткий зміст звіту

Не порожнє

comments

Коментар до звіту

Може бути порожнім

approval_status

Статус погодження звіту

Не порожнє, значення за замовчуванням 'Чернетка'

created_by_detective_id

Ідентифікатор детектива, який створив звіт

Зовнішній ключ для зв'язку з сутністю detective (id), не порожнє

Suspect (Підозрюваний)

id

Ідентифікатор підозрюваного

Первинний ключ

first_name

Ім'я

Може бути порожнім, якщо задано — лише українські літери

last_name

Прізвище

Може бути порожнім, якщо задано — лише українські літери

father_name

По батькові

Може бути порожнім, якщо задано — лише українські літери

nickname

Псевдонім

Може бути порожнім

phone_number

Номер телефону

Може бути порожнім, якщо задано — формат +380XXXXXXXXX

Продовження таблиці 3.1

Ім'я атрибута

Призначення атрибута

Обмеження

date_of_birth

Дата народження

Може бути порожньою, якщо задано — значення $\leq$ поточна дата

region

Область проживання

Може бути порожньою, якщо задано — лише українські літери

city

Місто проживання

Може бути порожнім, якщо задано — лише українські літери та дефіс

street

Вулиця

Може бути порожньою, якщо задано — українські літери, пробіли та дефіс

building_number

Номер будинку

Може бути порожнім, якщо задано — цифри та символ /

apartment_number

Номер квартири

Може бути порожнім, якщо задано — значення > 0

height

Зріст

Може бути порожнім, але якщо задано, то разом із weight і значення > 0

weight

Вага

Може бути порожньою, але якщо задано, то разом із height і значення > 0

physical_description

Опис зовнішності

Може бути порожнім

prior_convictions

Відомості про попередні судимості

Може бути порожнім

approval_status

Статус погодження

Не порожнє, значення за замовчуванням 'Чернетка'

created_by_detective_id

Ідентифікатор детектива, який створив витрату

Зовнішній ключ для зв'язку з сутністю detective (id), не порожнє

3.2 Формалізація зв'язків між сутностями

У спроектованій базі даних для предметної області «Детективне агентство» використовуються два основні типи зв'язків між сутностями: один-до-багатьох та багато-до-багатьох. Зв'язки типу один-до-одного в межах даної моделі окремо не реалізовувалися, оскільки структура предметної області не вимагала введення таких залежностей.

Розглянемо зв'язок між справами та доказами. Одна справа може містити кілька доказів, і водночас один доказ може бути пов'язаний лише з однією справою в межах поточної логіки системи через окрему асоціативну сутність. Для формального подання цього зв'язку використано проміжну таблицю `case_evidence`, до якої включено первинні ключі таблиць `case` та `evidence`. У цій таблиці поля `case_id` та `evidence_id` виступають зовнішніми ключами, а їх комбінація утворює складений первинний ключ. Додатково в асоціативній таблиці зберігається атрибут `approval_status`, що дає змогу фіксувати стан погодження доказу в межах конкретної справи.

Аналогічним чином реалізовано зв'язок між справами та підозрюваними. Одна справа може містити декілька підозрюваних, а один підозрюваний може бути пов'язаний із конкретною справою через асоціативну сутність `case_suspect`. Для формалізації такого зв'язку створено проміжну таблицю `case_suspect`, яка містить зовнішні ключі `suspect_id` та `case_id`. Їх поєднання є складеним первинним ключем таблиці. Крім цього, у таблиці зберігаються додаткові атрибути `is_interrogated`, `alibi` та `approval_status`, що характеризують участь підозрюваного саме в межах певної справи. Наявність додаткових полів у цій таблиці дозволяє фіксувати не лише сам факт належності підозрюваного до справи, а й окремі обставини його участі в розслідуванні.

Зв'язки типу один-до-багатьох у моделі також представлені досить широко. Наприклад, один клієнт може мати багато справ, але кожна справа належить лише одному клієнтові. Для реалізації цього зв'язку ідентифікатор клієнта додано до таблиці `case` у вигляді зовнішнього ключа `client_id`. Подібним є зв'язок між детективом і справами: один детектив може бути призначений на багато справ, тоді як кожна справа може мати лише одного відповідального детектива. Для цього у таблиці `case` передбачено зовнішній ключ `detective_id`. Разом із цим у моделі зафіксовано зв'язки між детективом і матеріалами, що створюються в процесі розслідування. Зокрема, сутності доказів, підозрюваних, витрат і звітів містять посилання на детектива, який створив відповідний запис. Такий підхід дає змогу визначати не лише належність матеріалів до справи, а й відповідального користувача, який вніс ці дані до системи.

Так само за принципом один-до-багатьох реалізовано зв'язок між типом справи та справами. Один тип справи може використовуватися для багатьох записів у таблиці `case`, але кожна справа належить лише до одного типу. У зв'язку з цим у таблицю `case` включено зовнішній ключ `case_type_id`, що посилається на таблицю `case_type`.

Окремо слід розглянути зв'язки між справами та витратами, а також між справами та звітами. Одна справа може містити багато витрат і багато звітів, тоді як кожна витрата і кожен звіт належать лише одній конкретній справі. Для формалізації цих залежностей у таблицях `expense` та `report` додано зовнішній ключ `case_id`, який посилається на таблицю `case`. Такий підхід дає змогу зберігати всі супровідні матеріали розслідування у прив'язці до відповідної справи.

Одночасно це забезпечує цілісність даних і дозволяє швидко отримувати всі пов'язані записи в межах однієї справи.

Отже, формалізація зв'язків у базі даних побудована відповідно до логіки предметної області. Зв'язки один-до-багатьох реалізовано шляхом додавання зовнішнього ключа до таблиці залежної сутності, а зв'язки багато-до-багатьох — за допомогою окремих асоціативних таблиць, які, крім ключових полів, містять також додаткові атрибути, необхідні для збереження службової інформації. Така структура забезпечує цілісність даних, коректне подання взаємозв'язків між сутностями та зручність подальшої роботи із системою.

3.3 Концептуальна модель предметної області у вигляді ER-діаграми

ER-діаграма для предметної області «Приватне детективне агентство» наведена на рис. 3.1.

Рисунок 3.1 – ER-діаграма для предметної області «Приватне детективне агентство»

4 ПРОГРАМНА МОДЕЛЬ ЗАСТОСУНКУ

Модель взаємодії рівня бізнес-логіки в інформаційній системі «Детективне агентство» представлена компонентами, які забезпечують обробку дій користувачів, перевірку їхніх повноважень та взаємодію з базою даних. Оскільки система реалізована як веб-застосунок із використанням Razor Pages, обробка запитів користувача відбувається через відповідні сторінки та їхні PageModel-класи. Саме вони приймають запити від користувача, виконують необхідні дії над даними та повертають результат у вигляді оновленої веб-сторінки.

У межах застосунку користувачі взаємодіють із системою через інтерфейсні елементи: кнопки, форми, таблиці, поля пошуку та посилання для переходу між сторінками. Після виконання певної дії, наприклад створення клієнта, додавання доказу, редагування витрати або надсилання запису на погодження, відповідний PageModel обробляє запит і звертається до рівня даних. Для цього використовується екземпляр класу DbContext, який забезпечує доступ до таблиць бази даних через відповідні властивості-множини.

Для взаємодії з базою даних у проєкті використовується Entity Framework Core. Ця технологія дозволяє працювати з даними не безпосередньо через SQL-запити, а через класи-моделі, які відповідають таблицям бази даних. Наприклад, таблиці client, detective, case, evidence, suspect, expense та report представлені у програмному коді відповідними класами. Зміни, які користувач виконує через інтерфейс, обробляються застосунком і передаються до бази даних за допомогою DbContext.

Загальний вигляд взаємодії між сторінкою, PageModel, DbContext та класами-моделями доцільно подати у вигляді UML-діаграми. На такій діаграмі потрібно показати, що користувач взаємодіє з View, після чого запит обробляється PageModel або контролером, а далі через DbContext відбувається звернення до моделей та таблиць бази даних. У результаті така схема взаємодії забезпечує послідовний перехід від дій користувача на рівні інтерфейсу до обробки логіки застосунку та збереження змін у базі даних.

Рисунок 4.1 – UML-діаграма взаємодії контролеру з класом DbContext

Як видно з рисунка 4.1, DbContext є проміжною ланкою між бізнес-логікою застосунку та базою даних. Він надає доступ до наборів сутностей, які відповідають таблицям бази даних. Завдяки цьому система може отримувати, створювати, змінювати та видаляти записи відповідно до прав користувача. Наприклад, адміністратор може працювати з клієнтами, справами та погодженням матеріалів, а детектив – лише з призначеними йому справами та пов'язаними з ними записами. Крім того, використання DbContext забезпечує централізовану організацію доступу до даних, що спрощує взаємодію між окремими компонентами системи. Це дозволяє узгоджено працювати з різними сутностями предметної області та підтримувати цілісність даних під час виконання основних операцій.

Кожна таблиця бази даних транслюється у відповідний клас моделі. У такому класі описуються властивості, які відповідають полям таблиці. Наприклад, таблиця client має поля id, first_name, last_name, email, phone_number, city, region та інші, а в програмному коді їм відповідають властивості класу Client. Аналогічно таблиця case представлена класом Case, таблиця detective – класом Detective, таблиця evidence – класом Evidence тощо.

Рисунок 4.2 – Порівняння вихідної таблиці та трансльованого класу моделі

Рівень представлення інформаційної системи являє собою ієрархію веб-сторінок, через які користувачі взаємодіють із системою. Він забезпечує зручний та інтуїтивно зрозумілий інтерфейс для виконання основних операцій і відображення даних у структурованому вигляді. Доступ до сторінок залежить від ролі авторизованого користувача, що дозволяє обмежити функціональність відповідно до його повноважень та забезпечити безпеку обробки інформації. У системі передбачено дві основні ролі: адміністратор і детектив. Після авторизації користувач потрапляє на відповідну панель, де відображаються лише ті розділи та елементи керування, які доступні його ролі, що спрощує навігацію та зменшує ймовірність помилкових дій. Така організація рівня представлення дозволяє узгодити структуру інтерфейсу з функціональними обов'язками кожної категорії користувачів. Крім того, поділ сторінок за ролями сприяє більш чіткому розмежуванню доступу до даних і підвищує зручність повсякденної роботи із системою.

Адміністратор має доступ до панелі адміністратора, з якої може перейти до сторінок роботи з клієнтами, детективами, справами, доказами, витратами, підозрюваними, звітами та типами справ. Інтерфейс адміністратора побудований таким чином, щоб забезпечити швидкий доступ до ключових даних та можливість централізованого керування всіма процесами системи. Для частини сутностей адміністратор може створювати, переглядати або редагувати записи, а для матеріалів, створених детективом, основною функцією адміністратора є погодження або відхилення. Додатково адміністратор здійснює контроль за актуальністю інформації, коректністю введених даних та узгодженістю між різними сутностями. Відповідна структура сторінок дає змогу швидко переходити між підсистемами та виконувати необхідні операції без зайвих проміжних дій. Таким чином, у підсистемі адміністратора реалізовано загальне керування системою та контроль за роботою з матеріалами справ.

Детектив після авторизації переходить на власну панель, де має доступ лише до тих справ, на які його призначено. Інтерфейс детектива орієнтований на виконання практичних завдань у межах конкретних справ і не перевантажений зайвими функціями. Він не може самостійно

створювати або закривати справи, оскільки це належить до повноважень адміністратора. Основна робота детектива полягає у супроводі призначених справ: перегляді деталей справи, додаванні доказів, підозрюваних, витрат і звітів, а також надсиланні цих записів на погодження адміністратору. Побудова інтерфейсу для цієї ролі спрямована на те, щоб зосередити увагу користувача саме на матеріалах розслідування та пов'язаних із ними діях. Такий підхід забезпечує чітке розмежування відповідальності та підвищує контрольованість процесів у системі.

Повна ієрархія веб-сторінок для користувачів представлена на рисунку 4.3. Вона демонструє, які сторінки доступні користувачеві після авторизації та які дії він може виконувати в межах кожної підсистеми залежно від ролі в ІС. Крім того, дана ієрархія відображає логічну структуру навігації, що дозволяє користувачеві швидко переходити між розділами та ефективно виконувати необхідні операції в межах системи. З огляду на це, рівень представлення забезпечує не лише відображення даних, а й логічну організацію взаємодії користувача з функціональними можливостями застосунку.

Рисунок 4.3 – Ієрархія форм підсистеми користувачів ІС «Приватне детективне агентство»

5 ВИБІР ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ ДЛЯ СТВОРЕННЯ ІНФОРМАЦІЙНОЇ СИСТЕМИ

Для розробки інформаційної системи для предметної області «Детективне агентство» обрано такі інструменти:

мова програмування C#;

технологія Razor Pages;

система управління базами даних PostgreSQL;

технологія Entity Framework Core;

бібліотека jQuery;

бібліотека Dapper.

Існує багато різних систем управління базами даних, які можуть бути використані для реалізації інформаційної системи.

Для даного курсового проекту обрано PostgreSQL, оскільки ця система має відкритий вихідний код, підтримує роботу зі складними структурами даних, забезпечує надійне зберігання інформації та надає широкі можливості для підтримки цілісності бази даних. Серед її переваг можна виділити підтримку первинних і зовнішніх ключів, унікальних обмежень, обмежень NOT NULL, а також інших механізмів, які дають змогу гарантувати коректність збережених даних. Крім того, PostgreSQL добре підходить для побудови масштабованих інформаційних систем і підтримує такі функціональні можливості, як вкладені запити, уявлення та інші засоби роботи з реляційними даними.

Для реалізації серверної частини застосунку обрано мову програмування C#, оскільки вона є сучасною об'єктно-орієнтованою мовою, що широко використовується для створення надійних програмних систем. Використання C# дало змогу реалізувати логіку роботи застосунку, взаємодію з базою даних та обробку запитів користувачів.

Як технологію побудови веб-інтерфейсу використано Razor Pages. Такий підхід дозволяє зручно організувати структуру сторінок застосунку, поєднати логіку обробки запитів із відповідними сторінками та спростити розробку інтерфейсу користувача. Це є доцільним для створення веб-застосунку з чітко визначеними сторінками для різних категорій користувачів.

Для взаємодії застосунку з базою даних використано Entity Framework Core. Ця технологія дає змогу працювати з даними на рівні об'єктів мови програмування, спрощує реалізацію доступу до бази даних, підтримує міграції та полегшує супровід структури бази даних у процесі розробки.

Для реалізації окремих елементів клієнтської взаємодії використано бібліотеку jQuery. Її застосування дало можливість спростити роботу з елементами сторінки, обробку подій і реалізацію окремих динамічних елементів інтерфейсу. Таким чином, обраний набір інструментів забезпечує можливість реалізації повноцінної інформаційної системи для детективного агентства, яка підтримує роботу з базою даних, обробку користувацьких запитів і зручну взаємодію з даними через веб-інтерфейс.

Для реалізації окремих сценаріїв роботи з базою даних, що потребують більш гнучкого керування SQL-запитами, у проєкті використано бібліотеку Dapper. На відміну від Entity Framework Core, який застосовувався переважно для виконання базових CRUD-операцій над основними сутностями системи, Dapper використовувався для складніших вибірок, агрегатних обчислень, формування зведених даних і виконання спеціалізованих SQL-запитів.

Застосування Dapper дозволило напряду працювати з SQL та отримувати результати запитів з мінімальними накладними витратами, що є доцільним у випадках, коли необхідно побудувати більш складну логіку вибірки даних або підвищити продуктивність виконання окремих операцій.

6 СТВОРЕННЯ БАЗИ ДАНИХ

Розглянемо запит на створення таблиці Client:

Лістинг 6.1 – Запит на створення таблиці "Client"

Створення таблиці здійснюється за допомогою оператора CREATE TABLE, після якого задається її назва та перелік полів. Для кожного стовпця визначається тип даних і відповідні обмеження, що забезпечують коректність і узгодженість даних. У таблиці client поле client_id виконує роль первинного ключа і має тип serial, що дозволяє автоматично генерувати унікальні значення для кожного запису. Більшість полів, таких як first_name, last_name, city, region, street, email і phone_number, визначені з обмеженням NOT NULL, що означає обов'язковість їх заповнення.

Для контролю введених даних застосовано обмеження CHECK: apartment_number допускає лише додатні значення або може залишатися порожнім, date_of_birth не може перевищувати

поточну дату, а `registration_date` – поточний момент часу. Текстові поля перевіряються регулярними виразами: імена містять лише українські літери, поля `city`, `region` і `street` мають відповідний формат, а `building_number` допускає тільки цифри та символ `"/`. Для контактних даних передбачено перевірки: `phone_number` відповідає формату `(+380...)`, а `email` – стандартному формату електронної пошти.

Лістинг 6.2 – Запит на створення таблиці "Case"

Таблиця `case_type`, `client` та `detective` є батьківськими по відношенню до таблиці `case`, а таблиця `case` – дочірньою. У ній поля `case_type_id`, `client_id` та `detective_id` визначені як зовнішні ключі (REFERENCES), що забезпечує зв'язок із відповідними таблицями та гарантує цілісність даних між ними.

7 ЗАПИТИ ДО БАЗИ ДАНИХ ДЛЯ ВИРІШЕННЯ ПОСТАВЛЕНИХ ЗАДАЧ

У реалізації поставлених задач – операції виконуються переважно через Entity Framework Core (AdminService / DetectiveService); еквівалент показано як SQL. Також використано бібліотеку Dapper для виконання складніших операцій та агрегацій.

7.1 Задачі адміністраторів

1. Для розв'язання задач A1, A5, A9, A14 використовуються запити типу INSERT INTO таблиця (стовпець1, стовпець2, ...) VALUES (значення1, значення2, ...) з підстановкою відповідних імен таблиць і стовпців. При цьому для розв'язання задачі A1 замість параметра таблиця підставляється ім'я таблиці `client`, для розв'язання задачі A5 – ім'я таблиці `detective`, для розв'язання задачі A9 – ім'я таблиці `"case"`, а для розв'язання задачі A14 – ім'я таблиці `case_type`. Значення полів передаються з відповідних елементів користувацького інтерфейсу.

Наприклад, для розв'язання задачі A1 логічний запит має вигляд:

2. Для розв'язання задач A2, A6, A11, A15 використовуються запити типу DELETE FROM таблиця WHERE id = значення або UPDATE таблиця SET стовпець = NULL WHERE умова з підстановкою відповідних імен таблиць, стовпців та умов. При цьому для розв'язання задачі A2 замість параметра таблиця підставляється ім'я таблиці `client`, для розв'язання задачі A6 – ім'я таблиці `detective` з попереднім відв'язуванням детектива від справ, для розв'язання задачі A11 – ім'я таблиці `"case"` із присвоєнням полю `detective_id` значення NULL, а для розв'язання задачі A15 – ім'я таблиці `case_type`. Значення ідентифікатора запису, що видаляється або оновлюється, передається з відповідного елемента керування користувацького інтерфейсу. Наприклад, для задачі A2 використовується запит:

```
DELETE FROM client
```

```
WHERE id = {client_id};
```

3. Для розв'язання задач A4, A8, A10, A13, A16 використовуються запити типу UPDATE таблиця SET стовпець = значення [, ...] WHERE id = значення з підстановкою відповідних імен таблиць,

стовпців та нових значень. При цьому для розв'язання задачі A4 замість параметра таблиця підставляється ім'я таблиці client, для розв'язання задачі A8 – ім'я таблиці detective, для розв'язання задачі A10 – ім'я таблиці "case" з оновленням поля detective_id, для розв'язання задачі A13 – ім'я таблиці "case" з оновленням поля deadline_date, а для розв'язання задачі A16 – ім'я таблиці case_type. Наприклад, для задачі A10 логічний запит має вигляд:

4. Для розв'язання задач A3, A7, A12, A18 необхідні запити типу SELECT стовпець1, стовпець2, ... FROM таблиця [JOIN таблиця2 ON умова] WHERE умова з підстановкою відповідних імен таблиць, стовпців і умов. При цьому для розв'язання задачі A3 використовується вибірка з таблиці client з додатковим отриманням пов'язаних справ, для розв'язання задачі A7 – вибірка з таблиці detective з додатковими агрегатами по справах, для розв'язання задачі A12 – вибірка з таблиці "case" з приєднанням таблиці detective, а для розв'язання задачі A18 – вибірка з таблиці report за ідентифікатором звіту. Наприклад, для задачі A12 може бути використаний запит:
5. Для розв'язання задач A17, A19, A20, A21, A22, A23, A24, A26 використовуються запити типу SELECT стовпець1, стовпець2, ... FROM таблиця [JOIN ...] WHERE умова ORDER BY стовпець з підстановкою відповідних імен таблиць, стовпців, умов фільтрації та полів сортування. При цьому для розв'язання задачі A17 формується список звітів по конкретній справі, для задачі A19 – список справ за заданим статусом, для задачі A20 – список справ, відсортованих за наближенням до дедлайну, для задачі A21 – список відкритих справ, найближчих до дедлайну, для задачі A22 – перелік справ конкретного клієнта, для задачі A23 – список клієнтів за періодом реєстрації, для задачі A24 – інформація про навантаження детектива, а для задачі A26 – перелік справ із простроченим дедлайном. Наприклад, для задачі A21 використовується запит:
6. Для розв'язання задачі A25 створена функція get_average_case_cost(_detective_id integer), яка обчислює середню вартість справ, що вів конкретний детектив, з урахуванням базової вартості типу справи та додаткових витрат. Для звернення до неї використовується запит:

SELECT *

FROM get_average_case_cost({id_детектива});
7. Для розв'язання задачі A27 використовується запит типу SELECT стовпець1, стовпець2, агрегатна_функція(...) FROM таблиця LEFT JOIN таблиця2 ON умова GROUP BY ... ORDER BY ..., який дозволяє отримати список клієнтів із кількістю незавершених справ. При цьому використовуються таблиці client та "case", а до вибірки включаються лише ті справи, статус яких відмінний від значення «Закрито». Логічний запит має вигляд:
8. Для розв'язання задачі A28 створена функція get_average_case_cost(_detective_id integer) (див. додаток В), яка викликається при виконанні запиту:
9. Для розв'язання задачі A29 створено представлення detective_ranking, яке використовується для формування зведеної інформації про роботу детективів за поточний рік. Представлення містить ідентифікатор детектива, його ім'я, відсоток закритих справ, кількість прострочених

справ, середню вартість справ, а також кількість пов'язаних доказів і підозрюваних. Для отримання даних використовується запит:

10. Для розв'язання задачі A30 створено представлення `first_time_clients`, яке формує список клієнтів, що звернулися вперше у поточному році. Для звернення до нього використовується запит:
11. Для розв'язання задачі A31 використовується запит типу `SELECT ..., ROW_NUMBER() OVER (...) FROM таблиця LEFT JOIN таблиця2 ON умова GROUP BY ... ORDER BY ...`, який дозволяє отримати рейтинг найпопулярніших типів справ. При цьому використовуються таблиці `case_type` та `"case"`. Запит має вигляд:
12. Для розв'язання задачі A32 використовується запит типу `SELECT ..., ROW_NUMBER() OVER (...) FROM таблиця LEFT JOIN таблиця2 ON умова GROUP BY ... ORDER BY ...`, який дозволяє отримати рейтинг клієнтів за кількістю незавершених справ. При цьому використовуються таблиці `client` та `"case"`, а до розрахунку включаються лише справи, статус яких відмінний від значення «Закрито». Запит має вигляд:
13. Для розв'язання задачі A33 використовується запит типу `SELECT ..., ROW_NUMBER() OVER (...) FROM таблиця LEFT JOIN таблиця2 ON умова GROUP BY ... ORDER BY ...`, який дозволяє отримати рейтинг типів справ, що найчастіше залишаються незавершеними. При цьому використовуються таблиці `case_type` та `"case"`, а до вибірки включаються лише справи, статус яких відмінний від значення «Закрито». Запит має вигляд:
14. Для розв'язання задачі A34 використовується запит типу `SELECT ..., ROW_NUMBER() OVER (...) FROM таблиця JOIN таблиця2 ON умова LEFT JOIN таблиця3 ON умова GROUP BY ... ORDER BY ...`, який дозволяє отримати рейтинг справ за кількістю пов'язаних витрат. При цьому використовуються таблиці `"case"`, `client`, `detective` та `expense`. Запит має вигляд:

7.2 Задачі детективів

1. Для розв'язання задач Д1, Д3, Д14, Д19 використовуються запити типу `SELECT стовпець1, стовпець2, ... FROM таблиця WHERE id = значення` з підстановкою відповідних імен таблиць, стовпців і умов. При цьому для розв'язання задачі Д1 замість параметра `таблиця` підставляється ім'я таблиці `"case"`, для розв'язання задачі Д3 – ім'я таблиці `client`, для розв'язання задачі Д14 – ім'я таблиці `suspect`, а для розв'язання задачі Д19 – ім'я таблиці `"case"`. Значення ідентифікатора запису передається з відповідного елемента користувацького інтерфейсу. Наприклад, для розв'язання задачі Д19 може бути використаний запит:

```
SELECT c.id, c.title, c.start_date, c.deadline_date
```

```
FROM "case" c
```

```
WHERE c.id = {case_id};
```

2. Для розв'язання задач Д2, Д6, Д15 використовуються запити типу UPDATE таблиця SET стовпець = значення [, ...] WHERE id = значення з підстановкою відповідних імен таблиць, стовпців і нових значень. При цьому для розв'язання задачі Д2 замість параметра таблиця підставляється ім'я таблиці "case", для розв'язання задачі Д6 – ім'я таблиці evidence, а для розв'язання задачі Д15 – ім'я таблиці suspect. Нові значення передаються з відповідних полів форми. Наприклад, для розв'язання задачі Д2 використовується запит:
3. Для розв'язання задач Д4, Д10, Д18 використовуються запити типу INSERT INTO таблиця (стовпець1, стовпець2, ...) VALUES (значення1, значення2, ...) з підстановкою відповідних імен таблиць, стовпців і значень. При цьому для розв'язання задачі Д4 замість параметра таблиця підставляється ім'я таблиці evidence, для розв'язання задачі Д10 – ім'я таблиці suspect, а для розв'язання задачі Д18 – ім'я таблиці report. Значення полів передаються з відповідних елементів користувацького інтерфейсу. Наприклад, для розв'язання задачі Д18 логічний запит має вигляд:
4. Для розв'язання задач Д7, Д8, Д11, Д12 використовуються запити типу INSERT INTO таблиця (...) VALUES (...) або DELETE FROM таблиця WHERE умова з підстановкою відповідних імен таблиць і значень. При цьому для розв'язання задачі Д7 замість параметра таблиця підставляється ім'я таблиці case_evidence, для розв'язання задачі Д8 – також ім'я таблиці case_evidence, для розв'язання задачі Д11 – ім'я таблиці case_suspect, а для розв'язання задачі Д12 – ім'я таблиці case_suspect. Значення ідентифікаторів справи, доказу або підозрюваного передаються з відповідних елементів інтерфейсу. Наприклад, для розв'язання задачі Д7 використовується запит:
5. Для розв'язання задач Д9, Д16, Д20 використовуються запити типу SELECT стовпець1, стовпець2, ... FROM таблиця1 JOIN таблиця2 ON умова WHERE умова ORDER BY стовпець з підстановкою відповідних імен таблиць, стовпців, умов фільтрації та полів сортування. При цьому для розв'язання задачі Д9 формується список доказів, прив'язаних до справи, для розв'язання задачі Д16 – список підозрюваних, прив'язаних до справи, а для розв'язання задачі Д20 – список незакритих справ. Наприклад, для розв'язання задачі Д9 використовується запит:
6. Для розв'язання задачі Д13 використовуються запити типу DELETE FROM таблиця WHERE умова. Спочатку видаляються зв'язки підозрюваного зі справами з таблиці case_suspect, після чого видаляється сам запис із таблиці suspect. Значення ідентифікатора підозрюваного передається з відповідного елемента керування користувацького інтерфейсу. Логічні запити мають вигляд:
7. Для розв'язання задач Д17, Д21, Д22, Д23, Д24 використовуються запити типу SELECT стовпець1, стовпець2, ... FROM таблиця [JOIN ...] WHERE умова ORDER BY стовпець з підстановкою відповідних імен таблиць, стовпців, умов фільтрації та полів сортування. При цьому для розв'язання задачі Д17 формується список усіх справ за заданим типом, для розв'язання задачі Д21 – список підозрюваних із заданими характеристиками, для розв'язання задачі Д22 – вибірка останнього створеного звіту по справі, для розв'язання задачі Д23 – пошук підозрюваних за місцем проживання, а для розв'язання задачі Д24 –

отримання статусів усіх справ, до яких прив'язаний конкретний підозрюваний. Наприклад, для розв'язання задачі Д22 використовується запит:

7.3 Створення тригерів

Тригерна функція `check_detective_availability` (див. додаток А2) виконує перевірку, чи не закріплений детектив за іншою активною справою на поточний момент. Для цього здійснюється запит до таблиці `case`, де відбираються всі незакриті справи, що вже розпочаті та ще не завершені.

Якщо така справа існує, функція повертає помилку, повідомляючи, що детектив зайнятий, а також вказує найближчу дату, коли він може звільнитися. У разі відсутності конфлікту запис дозволяється до вставки або оновлення.

Тригерна функція `get_average_case_cost` (див. додаток А1) використовується як допоміжний обчислювальний блок для показника середньої вартості справ конкретного детектива. Вона підключається в представленні `detective_ranking`: для кожного детектива з його ідентифікатора беруть середню вартість справ і виводять її в одному зі стовпців рейтингу разом із відсотком закритих справ, прострочками та лічильниками доказів і підозрюваних.

7.4 Створення процедур

Створення процедури здійснюється за допомогою команди `CREATE PROCEDURE`, після чого задається її ім'я та вхідні параметри. Далі визначається тіло процедури, у якому описується послідовність дій, що виконуються над даними.

У системі процедура `reassign_case` (див. додаток А3) призначена для перепризначення справи від одного детектива іншому. Вона приймає ідентифікатор детектива (`det_id`) і спочатку визначає його останню відкриту справу на основі дати початку.

Якщо відкритих справ немає, процедура завершує виконання та виводить відповідне повідомлення. У протилежному випадку виконується пошук іншого детектива, який працював зі справами такого ж типу. Для цього аналізуються записи в таблиці `case`, і обирається детектив із найбільш нещодавно завершеною справою. Якщо відповідний детектив знайдений, виконується оновлення запису – справа перепризначається новому детективу, а якщо ні – статус справи змінюється на «Призупинено».

У системі процедура `generate_payment_receipt` (див. додаток А4) використовується для формування платіжного звіту за конкретною справою. Вона приймає ідентифікатор справи (`p_case_id`), перевіряє наявність відповідного запису в таблиці `case`, після чого отримує основні відомості про справу: її назву, тип, базову вартість і повне ім'я клієнта шляхом об'єднання таблиць `case`, `case_type` і `client`. Далі обчислюється загальна сума витрат із таблиці `expense`, на основі чого визначається підсумкова сума до сплати. Отримані дані використовуються для формування PDF-документа, що містить основні відомості про справу та розрахунок суми до сплати. Після цього файл повертається користувачеві як документ типу `application/pdf` з назвою формату `PaymentReceipt_Case_.pdf`.

Процедуру `generate_payment_receipt` інтегровано в користувацький сценарій «Завантажити PDF-звіт по справі». Функціонал реалізовано на трьох рівнях: веб-рівні (Razor Pages), рівні бізнес-логіки (BLL) та рівні доступу до даних (DAL). На сторінці перегляду справи в модулі Cases/Details передбачено кнопку «Завантажити PDF». Після її натискання обробник передає ідентифікатор справи до сервісу формування звіту, де виконується виклик хранимої процедури PostgreSQL:

7.5 Створення представлень

У системі подання `detective_ranking` (див. додаток A5) інтегровано в адміністративний сценарій «Рейтинг детективів за поточний рік».

Функціонал описується на трьох рівнях: веб-рівень (Razor Pages), бізнес-логіка (BLL) і доступ до даних (DAL). На сторінці аналітики по детективах (модуль Admin, умовно Detectives/Ranking або блок на головній панелі адміністратора) відображається таблиця показників. Після завантаження сторінки обробник передає запит до сервісу звітності; у сервісі виконується звернення до бази у вигляді запиту до подання.

Отримані дані після цього перетворюються у зручну для відображення форму та передаються до сторінки. Завдяки цьому адміністратор отримує узагальнену інформацію про роботу детективів без необхідності формувати складні вибірки вручну.

Представлення `first_time_clients` (див. додаток A6) інтегровано в сценарій «Клієнти, які вперше звернулися в агентство в поточному році», що викликається наступним запитом:

8 ПРОГРАМНА РЕАЛІЗАЦІЯ ІНТЕРФЕЙСУ

Розглянемо програмну реалізацію інтерфейсу входу в систему. Для початку роботи користувачеві необхідно заповнити форму авторизації на сторінці `Login.cshtml`, вказавши ім'я користувача, пароль, а також за потреби обравши опцію «Запам'ятати мене». Після натискання кнопки «Увійти» формується HTTP POST-запит, який обробляється методом `OnPostAsync()` класу `LoginModel` (лістинг 8.1).

У межах цього методу спочатку виконується перевірка коректності введених даних. Якщо модель не проходить валідацію, сторінка повторно відображається користувачеві без подальшої обробки. Якщо ж дані заповнено коректно, формується об'єкт `LoginDto`, що містить введене ім'я користувача, пароль та значення опції запам'ятовування. Далі викликається сервіс автентифікації `AuthService.LoginAsync`, який перевіряє облікові дані користувача та визначає його роль у системі.

У разі успішної автентифікації створюється сесія користувача, зберігаються необхідні дані для подальшої роботи, а також формується cookie-автентифікація на основі `claims`. До списку `claims` додаються ім'я користувача, ідентифікатор, електронна пошта та роль. Після цього виконується перенаправлення на головну сторінку відповідної ролі: адміністратор потрапляє на сторінку `/Admin/Index`, а детектив – на сторінку `/Detective/Index`. Якщо вхід неуспішний, користувачеві виводиться повідомлення про помилку.

Лістинг 8.1, аркуш 3

Визначення ролі користувача в системі здійснюється на основі ролей PostgreSQL (лістинг 8.2). Для цього використовується окремий метод `ResolvePostgresRoleAsync`, який формує рядок підключення з переданими логіном і паролем користувача, встановлює з'єднання з базою даних та виконує SQL-запит для перевірки членства поточного користувача в ролях `admin` або `detective`. Якщо користувач належить до ролі адміністратора, метод повертає значення "Admin", якщо до ролі детектива – "Detective". У випадку неправильних облікових даних повертається `null`.

Лістинг 8.2 – Визначення ролі в PostgreSQL

Лістинг 8.2, аркуш 2

Після успішної авторизації подальша взаємодія користувача з системою відбувається відповідно до його ролі (лістинг 8.3). Адміністратор отримує доступ до сторінок керування клієнтами, детективами, справами, доказами, витратами, підозрюваними, звітами та типами справ. Детектив, у свою чергу, може працювати лише з призначеними йому справами та пов'язаними з ними матеріалами. В обох випадках користувач взаємодіє з системою через веб-сторінки, а відповідні `PageModel`-класи обробляють його дії, отримують або змінюють дані та повертають результат на екран.

Лістинг 8.3 – Авторизація, визначення ролі та розмежування доступу

Лістинг 8.3, аркуш 2

Наприклад, під час відкриття сторінки зі списком клієнтів адміністратор отримує дані з таблиці `client`, які завантажуються через `DbContext` і передаються у представлення для подальшого відображення (лістинг 8.4). Аналогічно формуються списки детективів, справ, доказів, витрат та інших сутностей. При створенні або редагуванні записів дані спочатку надходять із форми у властивості моделі сторінки, а потім, після перевірки, зберігаються у базі даних.

Лістинг 8.4 – Формування сторінки зі списком клієнтів

9 БЕЗПЕКА ІНФОРМАЦІЙНОЇ СИСТЕМИ

У системі безпека доступу реалізована за моделлю RBAC через поєднання механізмів PostgreSQL і веб-рівня застосунку. На боці БД використовуються групові ролі `admin` і `detective`, які створюються під час ініціалізації застосунку. Якщо ролі ще не існують, вони додаються як `NOLIGIN`, тобто як ролі-привілеї, а не окремі облікові записи для входу.

Лістинг 9.1 – Створення групових ролей `admin` та `detective` в PostgreSQL

Після створення ролям надаються привілеї на схему, таблиці, послідовності та службові enum-типи (див. табл. 9.1). Для адміністратора застосовується повний набір CRUD-операцій над основними сутностями, для детектива – обмежений доступ відповідно до його сценаріїв роботи. Наприклад, для таблиці справ детективу дозволено читання та оновлення, але не повне адміністрування.

Лістинг 9.2 – Надання базових привілеїв ролям `admin` та `detective` на схему та таблицю справ

Таблиця 9.1 – Привілеї ролей на об’єкти ІС «Приватне детективне агентство»

Об’єкти БД (таблиці)

Роль admin

(адміністратор)

Роль detective

(детектив)

case_type

CRUD

R

client

CRUD

R

Продовження таблиці 9.1

Об’єкти БД (таблиці)

Роль admin

(адміністратор)

Роль detective

(детектив)

detective

CRUD

R

case

CRUD

RU

evidence

CRUD

CRU

suspect

CRUD

CRU

case_evidence

CRUD

CRUD

case_suspect

CRUD

CRUD

expense

CRUD

CRU

report

CRUD

CRU

Автентифікація виконується через реальне підключення до PostgreSQL під логіном і паролем, які користувач вводить у формі входу. Після успішного підключення система перевіряє членство поточного користувача в ролях admin або detective через pg_has_role. pg_has_role – це вбудована функція PostgreSQL, яка повертає логічне значення (true/false) і дозволяє перевірити, чи має користувач певну роль (у цьому випадку – через режим member, тобто членство в ролі-групі). На основі результату цієї перевірки застосунок призначає внутрішню роль користувача.

Лістинг 9.3 – Визначення ролі користувача через перевірку членства в ролях PostgreSQL

Після визначення ролі на веб-рівні формується cookie-автентифікація з claims. Claims – це набір підтверджених атрибутів (тверджень) про користувача (наприклад, ім'я, ідентифікатор, email, роль), які записуються в автентифікаційний квиток (cookie) і надалі використовуються без повторної перевірки пароля на кожному запиті. Це потрібно для збереження стану входу та швидкого прийняття рішень авторизації в межах сесії. Далі доступ до сторінок і методів контролюється політиками авторизації, що напряду відповідає моделі розмежування прав у БД.

Лістинг 9.4 – Налаштування політик авторизації на веб-рівні застосунку

Отже, доступ захищено двома узгодженими шарами: PostgreSQL-контролем привілеїв (ролі та GRANT) і прикладним контролем доступу (claims + policy-based authorization). Policy-based authorization – це підхід, за якого правила доступу описуються у вигляді політик (наприклад, «лише користувачі з роллю Admin»), а потім централізовано застосовуються до сторінок, контролерів або окремих методів. Таке поєднання забезпечує коректне розділення повноважень адміністратора та детектива і безпечну роботу з даними відповідно до наданої ролі.

У системі – безпека доступу реалізована за моделлю RBAC через поєднання механізмів PostgreSQL і вебрівня застосунку. На рівні бази даних використовуються групові ролі admin і detective, які створюються під час ініціалізації системи. Якщо такі ролі ще не існують, вони додаються як NOLOGIN, тобто як ролі-привілеї, а не окремі облікові записи для входу.

Лістинг 9.5, аркуш 2

Після створення ролей їм надаються права доступу до схеми public, таблиць, послідовностей і службових enum-типів. Для ролі admin передбачено повний набір операцій створення, читання, оновлення та видалення даних, тоді як роль detective має обмежений доступ відповідно до своїх функціональних обов'язків.

Лістинг 9.6 – Надання повного набору привілеїв ролям admin та detective на об'єкти бази даних

10 ІНСТРУКЦІЯ КОРИСТУВАЧА

10.1 Інтерфейс адміністратора

При запуску програми користувач бачить вікно авторизації (рис. 10.1).

Рисунок 10.1 – Сторінка авторизації

Після вводу логіну та паролю та натискання кнопки «Вхід» програма встановлює з'єднання з сервером БД та виводить головну сторінку для користувача Адміністратор (рис. 10.2, а), або видає помилку про неможливість підключення (рис. 10.2, б).

Рисунок 10.2 (а) – Головна сторінка для користувача Адміністратор

Рисунок 10.2 (б) – Неуспішне підключення до бази даних

На головній сторінці – користувач може обрати необхідну йому таблицю та взаємодіяти із нею. Взаємодія буде відрізнятися в залежності від повноважень працівника. Для прикладу візьмемо дві таблиці – клієнти та докази, оскільки вони мають різний функціонал та принцип праці із ними.

Почнемо з сторінки клієнтів (рис. 10.3). Основну частину сторінки займає таблиця «Клієнти». У ній відображаються основні дані про кожного клієнта, зокрема ідентифікатор, ім'я, електронна пошта, телефон, місто та дата реєстрації. Таблиця дає змогу користувачеві швидко отримати загальне уявлення про наявні записи без необхідності відкривати кожен із них окремо.

Рисунок 10.3 – Таблиця «Клієнти»

На сторінці таблиці клієнтів одразу доступні всі основні функції для роботи з даними. У верхній частині сторінки розміщено поле пошуку, яке використовується для швидкого знаходження потрібного клієнта серед наявних записів. Це особливо зручно у випадку великої кількості даних, оскільки дозволяє не переглядати весь список вручну, а одразу відфільтрувати потрібний результат за введеним значенням.

Поруч розташована кнопка створення нового клієнта (рис. 10.4). Вона забезпечує перехід до окремої форми, у якій користувач може ввести всі необхідні дані для додавання нового клієнта до системи. Таким чином реалізується функція поповнення бази новими записами.

Рисунок 10.4 – Сторінка «Створити клієнта»

Функція пошуку (рис. 10.5) є важливою складовою сторінки, оскільки дає змогу оперативно знаходити потрібного клієнта за введеним значенням без необхідності переглядати всю таблицю вручну. Користувач може ввести частину імені, електронної адреси, номера телефону або іншого відомого параметра, після чого система відобразить лише ті записи, які відповідають запиту. Такий підхід значно пришвидшує роботу з даними, особливо коли кількість клієнтів у системі є великою, і робить процес пошуку більш зручним та ефективним.

Рисунок 10.5 – Демонстрація функції «пошук»

Для зручності роботи в таблиці реалізовано сортування за стовпцями (рис. 10.6). Це означає, що користувач може впорядковувати записи за певним полем, наприклад за іменем, містом або датою реєстрації. Такий механізм спрощує навігацію по даних і дозволяє швидше знаходити потрібну інформацію залежно від обраного критерію.

Рисунок 10.6 – Демонстрація сортування

Крім перегляду загального списку, система надає можливість перейти до детальної інформації про конкретного клієнта (рис. 10.7). Для цього достатньо натиснути на відповідний запис у таблиці. Після переходу відкривається сторінка окремого клієнта, де можна ознайомитися з повнішою інформацією про нього та, за потреби, виконати подальші дії.

Рисунок 10.7 – Таблиця «Деталі клієнта»

Тепер перейдемо до сторінки «докази». На сторінці таблиці доказів адміністратор отримує доступ до перегляду всіх наявних записів, а також до функцій пошуку, сортування та зміни статусу кожного доказу.

У таблиці відображаються основні відомості про доказ, зокрема його ідентифікатор, тип, опис, дата збору, регіон та поточний статус. Така структура дозволяє швидко оцінити зміст запису та його стан у межах робочого процесу.

Рисунок 10.8 – Таблиця «Докази»

На відміну від інших операцій із даними, у цій підсистемі (також в підсистемах «Підозрювані/Витрати/Звіти») адміністратор не створює і не видаляє записи про докази. Його роль полягає у

контролі та перевірці вже внесених даних. Саме тому для кожного запису передбачені дії схвалення або відхилення, які дозволяють адміністратору прийняти рішення щодо подальшого статусу доказу. Такий підхід забезпечує розмежування повноважень між користувачами та підтримує контроль за коректністю внесеної інформації. Окрім цього, подібна організація роботи дозволяє відокремити процес створення матеріалів від процесу їх адміністративної перевірки. Завдяки цьому в системі зберігається чіткий порядок опрацювання записів і підвищується надійність роботи з даними.

Якщо доказ має статус очікування, адміністратор може скористатися відповідними кнопками дії та або схвалити його, або відхилити (рис. 10.9). Після цього статус запису змінюється, а дія вважається завершеною. Для вже оброблених записів система відображає, що відповідна операція виконана, і повторне втручання не потрібне. Це дає змогу бачити, які докази ще очікують перевірки, а які вже були розглянуті адміністратором.

Рисунок 10.9 – Функціонал схвалення/відхилення

Отже, дана сторінка реалізує не функцію повного редагування сутностей, а механізм адміністративного контролю за доказами. Адміністратор може переглядати записи, знаходити потрібні докази, впорядковувати їх та приймати рішення щодо їх схвалення або відхилення, не змінюючи безпосередньо сам факт існування цих записів у системі.

10.2 Інтерфейс детектива

Після успішної авторизації користувач із роллю детектива потрапляє на головну сторінку (рис. 10.10), де відображаються доступні йому підсистеми для роботи з призначеними справами. На відміну від адміністратора, функціональність детектива зосереджена не на загальному керуванні системою, а на безпосередньому супроводі розслідувань, до яких він був призначений.

Рисунок 10.10 – Головна сторінка для користувача Детектив

На головній сторінці детектив може перейти до таблиці справ (рис. 10.11), а також до пов'язаних із ними підсистем, зокрема підозрюваних, доказів, витрат і звітів. Взаємодія з цими підсистемами обмежується лише тими записами, які належать до його справ, що забезпечує дотримання розмежування повноважень між різними категоріями користувачів.

Почнемо з таблиці справ. Основну частину сторінки займає таблиця «Справи». У ній відображаються справи, на які призначено поточного детектива. Користувач має можливість переглядати основну інформацію про кожну справу, ознайомлюватися з її змістом та переходити до детальнішого перегляду. При цьому детектив не має права змінювати саму справу, редагувати її основні параметри або закривати її, оскільки такі дії належать до повноважень адміністратора.

Рисунок 10.11 – Таблиця «Справи»

Таким чином, підсистема справ для детектива виконує функцію перегляду та навігації (рис. 10.12). Вона дає змогу бачити перелік призначених справ і працювати лише з тими матеріалами,

які безпосередньо стосуються цих справ.

Рисунок 10.12 – Таблиця пов'язаних матеріалів справи

Далі перейдемо до пов'язаних сутностей. У межах своїх справ детектив має можливість працювати з підозрюваними, доказами, витратами та звітами. На відміну від таблиці справ, у цих підсистемах користувач може створювати нові записи, переглядати вже наявні, редагувати їх, видаляти та прив'язувати до відповідної справи (рис. 10.13). Саме через ці сутності детектив наповнює систему інформацією, необхідною для супроводу розслідування.

Рисунок 10.13 – Таблиця «Докази»

Для кожної з таких підсистем реалізовано функцію створення нового запису (рис. 10.14). Це дозволяє детективу додавати нові докази, вносити інформацію про підозрюваних, фіксувати витрати, пов'язані з розслідуванням, а також формувати звіти за результатами виконаної роботи. Усі ці записи створюються в межах конкретної справи та надалі можуть бути використані в процесі її супроводу.

Рисунок 10.14 – Створення нового доказу

Окрім створення нових сутностей, детектив може редагувати вже існуючі записи, якщо потрібно уточнити або оновити внесену інформацію. Також передбачено можливість видалення записів у випадках, коли вони були додані помилково або втратили актуальність (рис. 10.15). Завдяки цьому користувач має змогу підтримувати матеріали справи в актуальному стані та коригувати їх у процесі роботи.

Рисунок 10.15 – Редагування та видалення записів

Важливою особливістю роботи детектива є можливість надсилати створені або оновлені записи на погодження (рис. 10.16). Після завершення внесення інформації користувач може ініціювати процедуру передання запису на розгляд адміністратору. Це стосується доказів, витрат, звітів та інших матеріалів, які потребують перевірки або підтвердження. Таким чином забезпечується поєднання оперативної роботи детектива з контролем з боку адміністратора.

Рисунок 10.16 – Надсилання запису на погодження

ВИСНОВКИ

У ході виконання курсового проєкту розроблено інформаційну систему «Детективне агентство», призначену для автоматизації основних процесів роботи агентства та впорядкування взаємодії з даними предметної області. У процесі роботи проаналізовано особливості діяльності детективного агентства, визначено основні категорії користувачів системи, їхні функції та повноваження, а також сформульовано вимоги до програмного застосунку.

У процесі розробки спроектовано базу даних, яка забезпечує зберігання інформації про працівників агентства, клієнтів, справи, підозрюваних, докази, витрати та звіти. Для всіх основних сутностей системи реалізовано функції перегляду, створення, пошуку, сортування,

редагування – залежно від ролі користувача. Окрему увагу приділено забезпеченню цілісності даних, захисту від несанкціонованого доступу та розмежуванню прав доступу між різними категоріями користувачів.

Для створення проєкту обрано трирівневу архітектуру, що дало змогу логічно розділити рівень представлення, бізнес-логіки та роботи з даними. Як шаблон проєктування використано MVC, що забезпечило зрозумілу структуру веб-застосунку, спростило організацію коду та дозволило розмежувати відповідальність між окремими компонентами системи. Для реалізації інформаційної системи використано мову програмування C#, технологію Razor Pages для побудови веб-інтерфейсу, систему управління базами даних PostgreSQL, Entity Framework Core для взаємодії з базою даних та jQuery для реалізації окремих елементів клієнтської взаємодії.

Додатково у процесі розробки реалізовано механізми перевірки коректності введених даних на рівні бази даних та прикладної логіки, що дозволило мінімізувати можливість виникнення помилок і забезпечити надійність збереження інформації. Використання обмежень, зовнішніх ключів і перевірок сприяє підтриманню узгодженості даних між пов'язаними сутностями. Також враховано питання масштабованості системи, що дає змогу в майбутньому розширювати її функціональність без суттєвих змін існуючої архітектури.

Окрему увагу приділено зручності користувацького інтерфейсу та логіці взаємодії з системою. Побудова інтерфейсу з урахуванням ролей користувачів дозволяє спростити роботу з системою, зменшити кількість зайвих дій та підвищити ефективність виконання основних операцій. Структурована навігація між сторінками та логічне групування функціональних можливостей забезпечують швидкий доступ до необхідної інформації та підвищують загальну зручність використання системи.

У результаті виконання курсового проєкту реалізовано підсистеми для роботи адміністратора та детектива. Адміністратор отримав інструменти для реєстрації працівників, контролю за справами, призначення детективів, прив'язки клієнтів до справ, а також перевірки та погодження матеріалів, що створюються в межах розслідувань. Детектив, у свою чергу, отримав можливість працювати з призначеними йому справами, додавати, редагувати та видаляти пов'язані записи про підозрюваних, докази, витрати та звіти, а також надсилати їх на погодження. Такий підхід забезпечує чітке розмежування повноважень між користувачами системи та реалізацію механізму контролю за внесеною інформацією.

Отже, поставлена мета курсового проєкту досягнута. Розроблена інформаційна система дає змогу автоматизувати ключові процеси роботи детективного агентства, підвищити впорядкованість збереження даних, спростити супровід детективних справ і покращити ефективність взаємодії користувачів із системою. Отриманий результат може слугувати основою для подальшого розширення функціональності, удосконалення інтерфейсу та розвитку системи відповідно до практичних потреб предметної області.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

Малахов Є.В., Проєктування баз даних та їх реалізація засобами стандартного SQL та PostgreSQL: Навч. посіб. для студ. вищих навч. закладів / Є.В. Малахов, О.А. Блажко, М.Г. Глава

// Одеса: БМБ, 2012. – 248 с.

Г. Гайна, Основи проектування баз даних. Навчальний посібник / Вид. «Кондор», 2018. – 204 с. 3.

PostgreSQL Documentation. – Режим доступу: <https://www.postgresql.org/docs/>

C# language documentation. – Режим доступу: <https://learn.microsoft.com/en-us/dotnet/csharp/>

Razor Pages in ASP.NET Core. – Режим доступу: <https://learn.microsoft.com/en-us/aspnet/core/razor-pages/?view=aspnetcore-10.0>

Entity Framework Core documentation. – Режим доступу: <https://learn.microsoft.com/en-us/ef/core/>

jQuery API Documentation. – Режим доступу: <https://api.jquery.com/>

ДОДАТОК А

Запити на створення тригерів, функцій та представлень

Функція (лістинг А.1) – обчислює середню вартість справ конкретного детектива за його ідентифікатором, враховуючи базову ціну типу справи та суму пов'язаних витрат.

Лістинг А.1 – функція для обчислення середньої вартості справ детектива

Функція (лістинг А.2) – аналізує таблицю case і перевіряє, чи має обраний детектив іншу активну (не закриту) справу на поточну дату. Якщо така справа існує, функція генерує EXCEPTION з датою найближчого завершення (deadline_date) і блокує призначення.

Таким чином, цей тригер реалізує бізнес-правило контролю навантаження детективів і запобігає одночасному призначенню одного детектива на кілька активних справ. Це дозволяє більш рівномірно розподіляти робоче навантаження між працівниками та уникати ситуацій перевантаження, які можуть негативно вплинути на якість виконання розслідувань. Крім того, використання такого механізму підвищує надійність системи управління справами та сприяє більш ефективному плануванню роботи агентства.